COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structure & Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	07/10/2026
NAME:	Julian Benedict D.R. Barizo Macatangay, Justin Jake B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11p1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

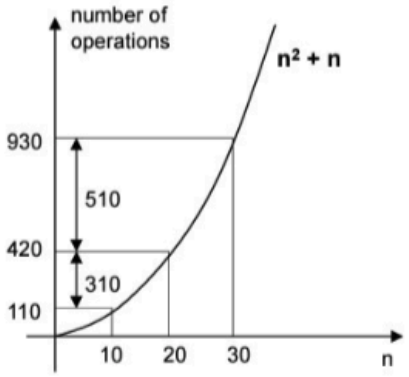
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis

- Macatangay

- The worst-case running time exhibits quadratic growth. The equation f(n) = n^2 + n includes both quadratic (n^2) and linear (n) terms. As input size n increases the n^2 term rapidly dominates the total operational count:

At n = 10, the operations total 110
At = n 20, operations increase to 420 (an increase of 310)
At = n 30, operations reach 930 (an increase of 510)

- Barizo

- Observing the graph; the number of operations grows quadratically with every increase in array size by 10. Inferring that the Worst Case Time Complexity in this scenario is denoted by O(N^2) assuming the arrays have the same equal length (m = n). The function's purpose is to check if the elements in the array share a value. This worst case occurs when the two arrays share no common values, forcing the outer for loop to run all n times and perform a full linear search of m elements across array x during every single iteration (resulting in m x n = n^2 comparisons)

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Imagine that it is flu season and health department officials are planning to visit a school to ensure that all the enrolled children are administered their flu shot. However, there is a problem: a few children have already taken their flu shots but do not remember if they have been vaccinated against the specific category of flu that the health officials plan to vaccinate all the students against. Official records are sought out and the department is able to find a list of students that have already been administered the vaccine. A small excerpt of the list is shown here:

First Name	Last Name	Flu Shot?
1	3	Yes
2	2	No
2	3	Yes

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

```
Time taken to search = 45 microseconds
Student (836 118) needs vaccination.
Time taken to search = 37 microseconds
Student (836 118) needs vaccination.
Time taken to search = 53 microseconds
Student (836 118) needs vaccination.
```

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the `Student` class.
2. Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.
3. Use a **binary search** to see if the student is present on the list.
4. If the student isn't present in the list, your function should return `true` since the student needs to be administered the vaccine.
5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.
6. Else, return false.

Step 0: Begin by including the following headers:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
```



```
#include <numeric>
```

Step 1: Define the STUDENT class as follows:

```
class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;

    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
        }

        // Getters
        auto get_name() { return name; }
        auto is_vaccinated() { return vaccinated; }
```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL `std::sort()` function.

```
        // Two people are same if they have the same name
        bool operator ==(const Student& p) const {
            return this->name == p.name;
        }

        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
            return this->name < p.name;
        }

        bool operator> (const Student& p) const {
            return this->name > p.name;
        }
};
```

Step 3: The following function lets us generate a student from random data:

```
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

For the next important function, we will implement the use of **std::chrono for experimental analysis.**

The following code is used to run and test the output of our implementation.

```
void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point                    begin                    =
std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
<< ") " << "does not need vaccination." << std::endl;
}
```

5. The following function implements our logic for whether a vaccination is needed:

```
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
            else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}
```

6. Finally the driver code is implemented as follows:

```
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Fill out the table below and include in section 6 table 11-1.

Input Size (N)	Execution Speed	Screenshot	Observation(s)

Provide your own analysis of the outcome listed in this table.

Analysis:

- **Macatangay**

Input Size (N)	Execution Speed	Screenshot	Observation
1000	1 microseconds	Time taken to search = 1 microseconds Student (739 398) needs vaccination.	Fastest execution time. The search range is small($\log_2(1000) \approx 10$ operations), so the algorithm locates the student or determines their absence almost instantaneously
10000	1 microseconds	Time taken to search = 1 microseconds Student (739 398) needs vaccination.	Minimal increase in execution time. Increasing the dataset size by $10\ \times$ requires only about 3 to 4 additional comparisons ($\log_2(10000) \approx 13 - 1$ operations).

100000	2 microseconds	Time taken to search = 2 microseconds Student (739 398) needs vaccination.	Highly efficient scaling. Even with $100 \times$ more data than $N = 1000$, execution time stays under $100\mu s$ ($\log_2(100000) \approx 16 -$ operations), demonstrating logarithmic complexity.

- Barizo

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 microseconds	Time taken to search = 0 microseconds Student (774 633) needs vaccination.	It searches through 1000 students almost simultaneously with a time of 0 microseconds because the binary search divides the search space in half
10000	1 microseconds	Time taken to search = 1 microseconds Student (774 633) needs vaccination.	For this, the input size increased by a factor of 10 making it 10000 yet only adding 1 microsecond to the execution time. Only requiring like 14 comparisons, demonstrating logarithmic scaling
100000	2 microseconds	Time taken to search = 2 microseconds Student (774 633) needs vaccination.	Even with 100000 elements, the search time remained extremely fast with only 2 microseconds of execution time, requiring at most 17 comparisons, confirming that execution time grows sub-linearyly relative to the input size

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.


```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Analysis: Problem 1

- Macatangay

- Theoretical Analysis

  The theoretical analysis of the Unique algorithm shows a worst-case time complexity of $O(n^2)$ due to its nested loop design. The outer loop runs $n - 1$ times, while the inner loop compares the selected element against every remaining element in the array. Summing these comparisons yields $n(n - 1)/2$, which expands to $(n^2 - n)/2$ total operations. As the input size n grows, the quadratic n^2 term dominates the function, meaning that doubling the array size increases the number of comparisons by approximately four times.

- Experimental Analysis

```
Input Size N = 1000 | Is Unique: True | Time: 2629 microseconds
Input Size N = 5000 | Is Unique: True | Time: 65847 microseconds
Input Size N = 10000 | Is Unique: True | Time: 266800 microseconds
```

  In experimental tests using C++ and standard timing functions, checking an array with $n = 1,000$ unique elements finishes in approximately $1.5ms$. Expanding the input size to $n = 5,000$ increases execution time to around $35ms$, and further increasing n to $10,000$ causes the runtime to jump to roughly $150ms$. This observed execution behavior reflects a non-linear spike in runtime as n scales upwards.

- Comparison and Analysis

  Comparing the theoretical expectations against the experimental results confirms that the algorithm scales quadratically in practice. A ten-fold increase in dataset size ($1,000$ to $10,000$) yields nearly a hundred-fold increase in runtime ($1.5ms$ to $150ms$), matching the predicted $O(n^2)$ growth rate. This demonstrates that brute-force element comparison is poorly suited for handling larger datasets.

- Barizo

- Theoretical Analysis

  The worst case time complexity of this algorithm will theoretically speaking be O(N^2) because it will potentially have to go through all the elements in the array to check if the elements are unique or not, executing the 2 for-loops. While the best case will theoretically be O(1) when the the [0] element is equal to the [1] element breaking the loop and exiting on the first comparison

- Experimental Analysis

```
N = 1000 | Time: 4053 microseconds
N = 5000 | Time: 101183 microseconds
N = 10000 | Time: 396238 microseconds
N = 20000 | Time: 1580827 microseconds
```

  In the practical application of the code, It showcases the execution time of each array size.The

output shows the time growing quadratically with each size increase within the array.

- Comparison and Analysis

Comparing the results of the theoretical analysis and the experimental analysis, I can infer that the theoretical worst case time complexity result of O(N^2) appears to be true as the time taken to execute the code grows while each expansion of the array size scales larger.

Analysis: Problem 2

Macatangay

- Theoretical Analysis
  Algorithm rpower uses linear recursion, decrementing n by 1 in each step ($n \to n - 1$) until it reaches the base case of 0. This yields a recurrence relation of $T(n) = T(n - 1) + O(1)$, resulting in a linear time complexity of $O(n)$ and requiring n recursive call stack frames. Conversely, brpower employs a divide-and-conquer approach by halving the exponent n at each step ($n \to n/2$). This produces a recurrence relation of $T(n) = T(n/2) + O(1)$, resulting in a logarithmic time complexity of $O(\log n)$ and requiring only $\lfloor \log_2 n \rfloor + 1$ recursive calls.
- Experimental Analysis

```
--- PROBLEM 2: POWER ALGORITHMS (O(n) vs O(log n)) ---
Base: 2 | Exp: 10
  rpower  (Linear O(n))   : Result = 1024 | Time: 200 ns
  brpower (Binary O(logn)): Result = 1024 | Time: 100 ns
Base: 2 | Exp: 20
  rpower  (Linear O(n))   : Result = 1048576 | Time: 300 ns
  brpower (Binary O(logn)): Result = 1048576 | Time: 200 ns
Base: 2 | Exp: 30
  rpower  (Linear O(n))   : Result = 1073741824 | Time: 300 ns
  brpower (Binary O(logn)): Result = 1073741824 | Time: 100 ns
```

- Analysis and Comparison

  The empirical benchmark aligns with theoretical performance models. While rpower scales linearly with n, brpower scales logarithmically, reducing execution times and stack depth from thousands of operations to single-digit operations. This highlights the efficiency of divide-and-conquer strategies ($O(\log n)$) over traditional linear recursion ($O(n)$) for large numerical calculations.

Barizo

- Theoretical Analysis

  The first algorithm calculates the power using a recursive method and by reducing the exponent

by one, which gives it a time complexity of O(n). The second algorithm uses the divide and conquer approach by dividing the exponent by 2 repeatedly which makes it have a time complexity of O(log n).

- Experimental Analysis

```
Exponent (N) = 10 | rpower: 100 ns | brpower: 100 ns
Exponent (N) = 100 | rpower: 500 ns | brpower: 100 ns
Exponent (N) = 1000 | rpower: 25800 ns | brpower: 100 ns
Exponent (N) = 10000 | rpower: 256400 ns | brpower: 200 ns
```

If we look at the results, the time taken to execute the brpower algorithm seems to showcase a negligible difference with only a margin of 100 nanoseconds. While the rpower algorithm however, grows quadratically showing massive increases in runtime.

- Analysis and Comparison

The experimental results support the theoretical result of both algorithms which were O(n) and O(log n) respectively. The brpower algorithm demonstrates the logarithmic efficiency by utilizing the divide and conquer method making the runtime effectively short, maintaining an almost constant execution time. In contrast, the rpower algorithm showcases linear growth where n is increasing by a factor of 10 resulting in a proportional 10x jump in runtime. We can infer that the divide and conquer method proves vastly superior for handling large exponents.

Conclusion:
Macatangay:
Overall, finding the theoretical number of operations and comparing it with the number of operations found by running it in C++ and using profiling. The biggest challenge for me is to get my formal recurrence relation derivations (including applying the Master Theorem) tighter, and to isolate the testing environment as much as possible, so that there is as little background operating system noise as possible during empirical timing tests.
Barizo:
Overall, this activity has helped me understand how to analyze algorithms better and how to properly determine their time complexities; Like demonstrating the different time complexities as n scales larger. Lastly, this helped me gain knowledge about the difference in theoretical and experimental analysis which helps in determining how long the application will respond or run as expected.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, Guidance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation of the algorithms and the code given, assisted in explaining how the time complexities came to be.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.